

Documentación de Proyecto:
Sistema POS Autogestionado Inteligente

1. Nombre del Proyecto

TacOS (Tactical Ordering System) / Taquería Inteligente

2. El Problema y la Solución

El Problema (El dolor del mercado)

En los minimarkets de comida rápida y taquerías de alto flujo, se presentan tres cuellos de botella críticos durante las horas pico:

Saturación en el punto de toma de pedidos: Los clientes hacen filas largas solo para ordenar y pagar, lo que genera frustración, pérdida de clientes potenciales que se van al ver el local lleno, y errores humanos al tomar la orden de forma verbal ("Se me olvidó que lo quería sin cebolla").

Costos operativos elevados y lentitud: Mantener personal dedicado exclusivamente a tomar pedidos con papel y lápiz ralentiza la rotación de mesas y aumenta los costos fijos del negocio.

Falta de datos y estática comercial: Los administradores no saben con certeza qué insumos se van a agotar en el día, ni ofrecen sugerencias de compra personalizadas al cliente para aumentar el ticket promedio (venta sugestiva), perdiendo oportunidades masivas de ingresos.

Lo que se va a Resolver (El Impacto)

El proyecto resolverá esto eliminando los intermediarios en la toma de datos del pedido. Al delegar el menú, la personalización y el pago al propio dispositivo del cliente mediante un código QR físico en la mesa o barra, el negocio gana:

Fila Cero: El cliente se sienta y ordena de inmediato.

Eficiencia en Cocina: Los pedidos entran limpios, estandarizados y en tiempo real a una pantalla digital.

Optimización con IA: El sistema incrementa las ventas de forma automática sugiriendo acompañamientos dinámicos según el comportamiento del consumidor.

3. Descripción del Proyecto

El sistema es una plataforma SaaS (Software as a Service) B2B de punto de venta (POS) y visualización de cocina en tiempo real, diseñada específicamente para negocios de comida ágil. La plataforma se divide en una arquitectura de tres aplicaciones o vistas interconectadas:

La Aplicación del Cliente (Web Web-Responsive / PWA): Funciona sin descargas. Al escanear un código QR único pegado en la mesa o barra, el cliente accede a una interfaz fluida donde explora el menú, personaliza sus tacos (tipo de carne, salsas, extras), recibe sugerencias inteligentes de adición de productos por parte de la IA, ingresa sus datos básicos y realiza el pago (digital o en caja).

El KDS de Cocina (Kitchen Display System): Una interfaz optimizada para pantallas táctiles en la zona de preparación. Reemplaza las comandas de papel tradicionales por tarjetas digitales que se actualizan instantáneamente mediante WebSockets. Muestra los tiempos de preparación recomendados por la IA para asegurar que el pedido salga caliente y a tiempo.

El Panel Administrativo (Dashboard): Una plataforma de escritorio para el dueño del negocio, donde gestiona el inventario, configura el menú, cambia precios en tiempo real y visualiza las analíticas predictivas de la IA sobre tendencias de consumo y proyecciones de ventas semanales.

4. Alcance del Proyecto

El proyecto contempla el desarrollo de una plataforma web integral compuesta por un backend unificado y tres interfaces de usuario diferenciadas. El sistema operará bajo una arquitectura en tiempo real, garantizando que el flujo de información desde que el cliente escanea el código QR hasta que la cocina despacha el pedido ocurra sin retrasos perceptibles.

Fuera del Alcance (Lo que NO hará el sistema en su primera versión):

No se desarrollarán aplicaciones nativas para Android o iOS (se prioriza Web Responsive de carga rápida).

No incluirá módulos de facturación electrónica compleja (se integrará con pasarelas de pago y se emitirá un ticket de compra simple/recibo digital).

5. Requisitos Funcionales (RF)

Los requisitos funcionales definen las acciones específicas que el software debe ser capaz de ejecutar. Los dividiremos por módulos o vistas:

A. Módulo de la Vista del Cliente

RF-01: Lectura y Contexto de Código QR: El sistema debe identificar automáticamente la mesa o barra del minimarket mediante parámetros únicos embebidos en la URL del código QR.

RF-02: Visualización del Menú: El usuario podrá navegar por las categorías de alimentos (tacos, combos, bebidas, adiciones) de forma fluida.

RF-03: Personalización de Productos: El cliente podrá seleccionar especificaciones para sus platos (ej. Tipo de tortilla, término de la carne, exclusión de ingredientes como cebolla o cilantro).

RF-04: Motor de Venta Sugestiva (IA): El sistema presentará recomendaciones dinámicas de productos complementarios en el carrito basándose en la selección actual del usuario.

RF-05: Captura de Datos de Contacto: Antes de finalizar la orden, el sistema solicitará obligatoriamente el nombre del cliente y su número de teléfono (para notificaciones de entrega).

RF-06: Selección de Método de Pago: Permitirá al cliente elegir entre pago digital inmediato (tarjeta de crédito/débito, pasarela local) o pago en efectivo/caja directamente en el local.

B. Módulo de la Vista del Vendedor (Cocina / KDS)

RF-07: Recepción de Pedidos en Tiempo Real: La interfaz de cocina deberá listar las nuevas comandas instantáneamente al ser confirmadas por el cliente, emitiendo una alerta sonora y visual.

RF-08: Gestión de Estados del Pedido: El personal de cocina podrá cambiar el estado de una orden a través de un tablero interactivo (Estados: En Cola -> Preparando -> Listo para Entregar).

RF-09: Desglose de Especificaciones: Las tarjetas de pedido deben resaltar visualmente las modificaciones o notas especiales hechas por el cliente (ej. "SIN PICANTE" en color rojo).

RF-10: Estimación de Tiempos (IA): El sistema calculará y mostrará en pantalla un temporizador inteligente con el tiempo estimado de preparación óptimo para cada pedido.

C. Módulo del Panel Administrativo (Dashboard)

RF-11: Gestión de Catálogo (CRUD): El administrador podrá crear, leer, actualizar y deshabilitar productos, categorías, precios y disponibilidad en tiempo real.

RF-12: Monitoreo de Ventas e Inventario: Visualización del flujo de caja diario, ticket promedio y alertas automáticas de insumos críticos.

RF-13: Analítica Predictiva (IA): El panel mostrará proyecciones automáticas de demanda de ingredientes y platos para la semana entrante basadas en el comportamiento histórico del negocio.

6. Requisitos No Funcionales (RNF)

Estas son las propiedades, cualidades y restricciones técnicas que debe cumplir el sistema para garantizar una experiencia óptima:

RNF-01: Latencia y Tiempo de Respuesta: Las actualizaciones de estado entre la vista del cliente y la cocina no deben superar los 2 segundos de retraso utilizando conexiones WebSocket.

RNF-02: Disponibilidad: El backend del SaaS debe garantizar un uptime del 99.9% para evitar caídas del servicio durante la operación comercial del restaurante.

RNF-03: Seguridad de Datos: El manejo de información de pago digital debe cumplir con los estándares PCI-DSS a través de la API de la pasarela de pagos seleccionada.

RNF-04: Portabilidad y Rendimiento Mobile: La aplicación del cliente debe pesar menos de 2MB en su carga inicial y ser completamente funcional en navegadores móviles estándar (Chrome, Safari) con conexiones móviles estándar (4G/5G).

6.1 Casos de usos

Para aterrizar los Requisitos Funcionales en interacciones reales del sistema, utilizaremos la narrativa de Casos de Uso. Esto modela el comportamiento exacto del software desde la perspectiva de cada actor (Cliente, Taquero/Vendedor, Administrador), definiendo el flujo principal y las alternativas en caso de errores.

1. Actor: Cliente (Dispositivo Móvil)

Caso de Uso 01: Realizar Pedido Auto-gestionado vía QR

Actor Principal: Cliente.

Precondiciones: El cliente debe estar físicamente en el local y haber escaneado el código QR de su mesa/barra con conexión a internet.

Flujo Principal (Camino Feliz):

El cliente escanea el código QR; el sistema detecta la URL y extrae automáticamente el identificador de la mesa (Ej: Mesa-3).

El sistema renderiza el catálogo de productos activo en pantalla.

El cliente navega por las categorías, selecciona los productos, especifica las cantidades y personaliza sus opciones (Ej: Tacos al pastor, sin cebolla, doble queso).

El cliente accede al carrito de compras.

Interacción de la IA: El sistema analiza la selección del carrito y muestra un banner flotante con productos sugeridos de alta afinidad (Venta sugestiva).

El cliente acepta o rechaza la sugerencia de la IA y presiona “Proceder al Pago”.

El sistema despliega el formulario donde el cliente ingresa su nombre, número telefónico y selecciona el método de pago (Pago en línea o Pago en caja).

El cliente presiona “Confirmar Pedido”.

El sistema procesa la transacción, guarda el pedido en la base de datos con estado PENDIENTE y dispara la alerta en tiempo real a la cocina vía WebSockets.

La pantalla del cliente cambia a una vista de espera que muestra el número de orden (Ej: #042) y el tiempo estimado por la IA.

Flujos Alternativos / Excepciones:

Excepción 3ª (Producto Agotado en Simultáneo): Si el cliente intenta añadir un producto que el administrador acaba de marcar como no disponible, el sistema muestra un mensaje flotante: “Lo sentimos, este ingrediente se ha agotado. Por favor selecciona otra opción”.

Excepción 7ª (Datos Incompletos): Si el cliente deja el campo de nombre o teléfono vacío, el botón “Confirmar Pedido” se mantiene inhabilitado y resalta los campos obligatorios en rojo.

2. Actor: Vendedor / Personal de Cocina (KDS – Pantalla Táctil)

Caso de Uso 02: Gestionar Flujo de Preparación en Cocina

Actor Principal: Taquero / Cocinero (Vendedor).

Precondiciones: La pantalla de la cocina debe estar encendida y conectada al canal de WebSockets del servidor.

Flujo Principal (Camino Feliz):

El backend recibe un nuevo pedido del cliente.

El sistema renderiza instantáneamente una nueva tarjeta virtual en la columna En Cola de la pantalla de la cocina y emite un sonido de notificación.

La tarjeta muestra el número de orden (#042), la ubicación (Mesa-3), el tiempo de preparación recomendado por la IA y la lista de platos con las modificaciones resaltadas (Ej: [SIN CEBOLLA] en texto llamativo).

El taquero toca la tarjeta para cambiar su estado a En Preparación. El sistema mueve la tarjeta de columna de forma fluida.

Al terminar de armar los tacos, el taquero toca nuevamente la tarjeta para marcarla como Listo para Entregar.

El sistema remueve la tarjeta del tablero de cocina y gatilla una notificación interna para el mesero o envía un mensaje automático al cliente.

Flujos Alternativos / Excepciones:

Excepción 2ª (Falta de Conectividad): Si la tablet de cocina pierde conexión a internet, el sistema muestra un banner persistente en la parte superior: “Sin conexión. Intentando reconectar... No cierre esta ventana”. Al recuperar señal, se sincronizan los pedidos perdidos de inmediato.

3. Actor: Administrador / Dueño del Minimarket (PC)

Caso de Uso 03: Gestión de Disponibilidad de Menú en Tiempo Real

Actor Principal: Administrador.

Precondiciones: El administrador debe haber iniciado sesión con sus credenciales de seguridad en el Dashboard.

Flujo Principal (Camino Feliz):

El administrador ingresa al módulo de “Inventario / Productos”.

Al percatarse de que la proteína de “Taco de Birria” se ha terminado en la cocina, busca el producto en la lista.

El administrador desmarca el interruptor (Switch Toggle) de Disponible en la fila del producto.

El administrador presiona “Guardar Cambios”.

El sistema actualiza el estado en la base de datos e invalida la caché del menú global.

De forma inmediata, cualquier cliente que escanee el QR o que tenga la aplicación abierta verá el producto bloqueado con la etiqueta “Agotado”, impidiendo que se generen pedidos imposibles de cumplir.

Caso de Uso 04: Consultar Analíticas Predictivas para Reabastecimiento

Actor Principal: Administrador.

Precondiciones: El sistema debe contar con un histórico mínimo de transacciones en la base de datos.

Flujo Principal (Camino Feliz):

El administrador accede a la sección de “Reportes e Inteligencia de Negocios”.

El sistema ejecuta el microservicio analítico en Python y procesa las series temporales cruzadas con variables de entorno.

El Dashboard renderiza un panel interactivo que muestra las proyecciones de venta para los próximos 7 días.

Interacción de la IA: El sistema despliega una tarjeta de recomendación textual explícita: “Se proyecta un incremento del 25% en la demanda de Tacos al Pastor este viernes debido al partido local y clima favorable. Se sugiere pre-ordenar 15kg adicionales de lomo de cerdo con tu proveedor mayorista”.

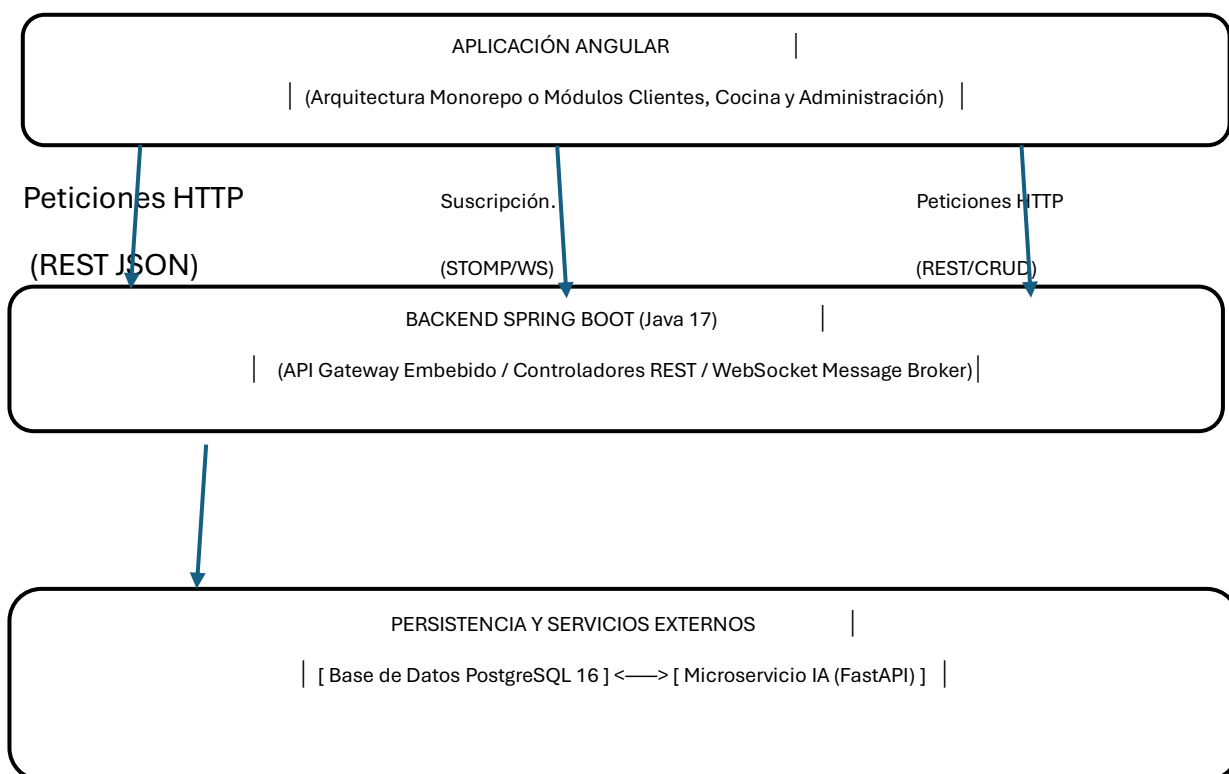
El administrador utiliza los datos para realizar sus compras de insumos semanales de forma precisa.

7. Arquitectura del Sistema

Para garantizar un rendimiento óptimo y cumplir con la latencia menor a 2 segundos (RNF-01), implementaremos una Arquitectura Hexagonal (Puertos y Adaptadores) en el backend con Spring Boot. Esto nos permite aislar la lógica del negocio (cálculo de totales, gestión de comandas) de tecnologías externas o cambiantes, facilitando las pruebas unitarias y la estabilidad ante actualizaciones.

7.1. Diagrama de Flujo y Comunicación Técnica

La comunicación entre los módulos de Angular y el servidor de Spring Boot se gestionará mediante dos protocolos diferenciados según la necesidad de velocidad:



Pilas Tecnológicas Detalladas (Tech Stack)

Para asegurar que el proyecto no sufra de conflictos de dependencias (“dependency hell”), nos basaremos estrictamente en versiones estables y con soporte a largo plazo (LTS).

8.1. Frontend: Angular 17+

Ecosistema: TypeScript como lenguaje nativo para asegurar contratos de datos estrictos (Interfaces) idénticos a los DTOs de Java.

Manejo de Estado y Concurrencia: RxJS para la gestión de flujos de datos asíncronos y reactivos, ideal para escuchar eventos del WebSocket en la cocina sin bloquear la interfaz.

Estilos: Tailwind CSS integrado en el flujo de compilación de Angular para un diseño web responsive ultraligero enfocado en dispositivos móviles.

8.2. Backend: Java 17 & Spring Boot 3.2.x

Elegimos la línea de Spring Boot 3.2 (o superior) ya que requiere Java 17 como mínimo absoluto, aprovechando características nativas como los Records (ideales para DTOs e inmutabilidad de datos de entrada) y mejoras críticas en el rendimiento de la memoria virtual.

Gestor de Dependencias: Maven (pom.xml).

Dependencias Core Altamente Compatibles (Spring Initializr):

Spring Boot Starter Web: Para exponer los controladores REST del menú y clientes.

Spring Boot Starter WebSocket: Provee la infraestructura para el protocolo STOMP sobre WebSockets para la comunicación en tiempo real con la cocina.

Spring Boot Starter Data JPA: Capa de abstracción sobre Hibernate para la persistencia.

PostgreSQL Driver: Conector nativo altamente optimizado.

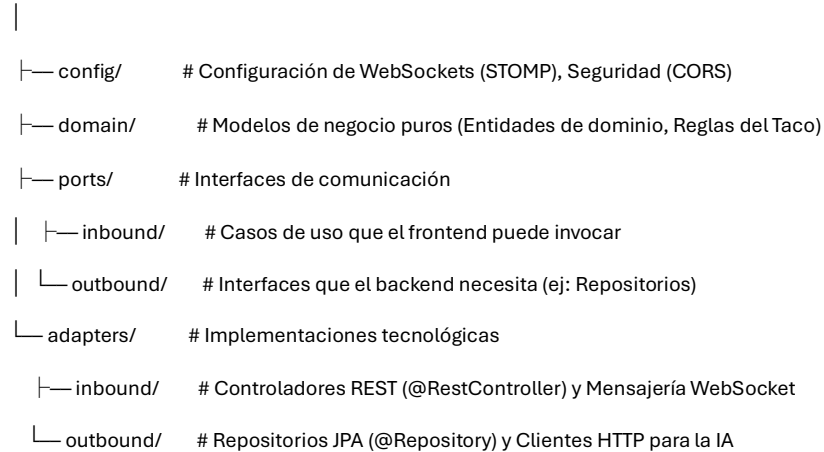
Lombok: Para la reducción de código repetitivo (Boilerplate) como Getters, Setters y Constructores.

9. Arquitectura de Carpetas del Proyecto (Estructura Base)

Para mantener la cohesión técnica y aplicar la arquitectura limpia propuesta, los proyectos se organizarán de la siguiente manera:

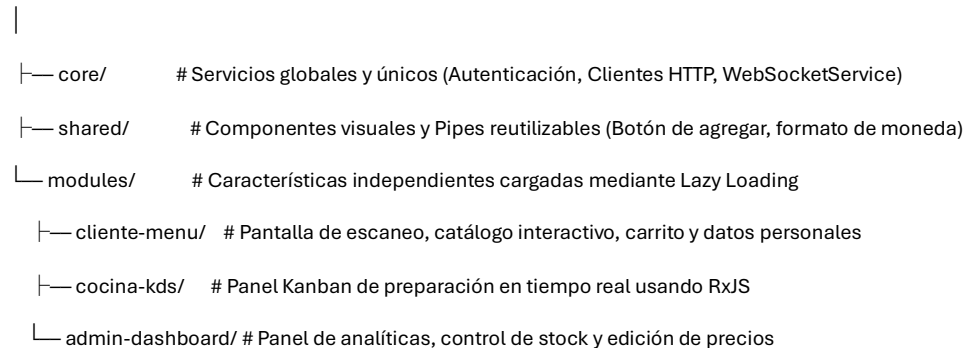
Backend: Estructura de Paquetes en Spring Boot

Src/main/java/com/tacos/api



Frontend: Estructura de Módulos en Angular

Src/app/



10. Diccionario de APIs (Contrato de Interfaz)

10.1. Módulo del Cliente (Angular \rightarrow Spring Boot via REST HTTP)

GET /api/v1/productos

- Descripción: Permite a la aplicación de Angular listar todo el menú disponible del minimarket cuando el cliente escanea el QR de la mesa.
- Códigos de Respuesta: 200 OK, 500 Internal Server Error.
- Respuesta Exitosa (JSON):

```
[
  {
    "id": "e3b0c442-98fc-11e3-a765-00a0c91e6bf6",
    "nombre": "Taco al Pastor",
    "descripcion": "Taco de cerdo marinado con piña, cilantro y cebolla",
    "precio": 4500.00,
    "categoria": "Tacos",
    "disponible": true
  },
  {
    "id": "f47ac10b-58cc-4372-a567-0e02b2c3d4e5",
    "nombre": "Quesadilla de Birria",
    "descripcion": "Tortilla de trigo grande con queso fundido y carne de birria",
    "precio": 12000.00,
    "categoria": "Quesadillas",
    "disponible": true
  }
]
```

POST /api/v1/pedidos

- Descripción: Envía la orden finalizada por el cliente desde el carrito de Angular hacia el servidor para su procesamiento y persistencia.
- Cuerpo de la Petición (Request Body):

```
{
  "nombreCliente": "Camilo Torres",
```

```

"telefonoCliente": "3004567890",

"identificadorMesa": "Mesa-3",

"notaCocina": "Los tacos al pastor sin cebolla y la salsa aparte.",

"metodoPago": "EFECTIVO_CAJA",

"items": [

  {

    "productoid": "e3b0c442-98fc-11e3-a765-00a0c91e6bf6",

    "cantidad": 3

  },

  {

    "productoid": "f47ac10b-58cc-4372-a567-0e02b2c3d4e5",

    "cantidad": 1

  }

]
}

```

Respuesta Exitosa (201 Created):

```

{

  "pedidoId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",

  "codigoOrden": 42,

  "estadoActual": "PENDIENTE",

  "totalNeto": 25500.00,

  "tiempoEstimadoPreparacion": "12 min",

  "createdAt": "2026-06-01T15:00:00Z"

}

```

10.2. Módulo de Cocina / KDS (Tiempo Real via WebSocket STOMP)

Para la sincronización instantánea de la tablet del taquero, Spring Boot expone un broker de mensajería asíncrono.

[SUBSCRIBE /topic/cocina/pedidos](#)

- Descripción: Al inicializar el módulo de cocina en Angular, la aplicación se suscribe a este canal.

- Payload Recibido (Cada vez que entra una orden nueva):

```
{
  "pedidoId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "codigoOrden": 42,
  "identificadorMesa": "Mesa-3",
  "notaCocina": "Los tacos al pastor sin cebolla y la salsa aparte.",
  "tiempoEstimadoIA": "12 min",
  "items": [
    {
      "nombreProducto": "Taco al Pastor",
      "cantidad": 3
    },
    {
      "nombreProducto": "Quesadilla de Birria",
      "cantidad": 1
    }
  ]
}
```

PATCH /api/v1/pedidos/{pedidoId}/estado

- Descripción: Permite al personal de cocina cambiar el estado del pedido conforme avanza la preparación.
- Cuerpo de la Petición (Request Body):

```
{
  "nuevoEstado": "PREPARANDO"
}
```

Nota: Los estados válidos restringidos por el backend serán: PENDIENTE, PREPARANDO, LISTO, ENTREGADO.

Respuesta Exitosa (200 OK):

```
{
  "pedidoId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "estadoActual": "PREPARANDO",
  "updatedAt": "2026-06-01T15:04:22Z"
}
```

10.3. Módulo de Administración (REST HTTP para Dashboard)

GET /api/v1/admin/metricas/hoy

- Descripción: Entrega los datos financieros e indicadores del día actual para los componentes gráficos de Angular.
- Respuesta Exitosa (200 OK):

```
{
  "ventasTotales": 452000.00,
  "pedidosAtendidos": 18,
  "ticketPromedio": 25111.11,
  "platoMasVendido": "Taco al Pastor",
  "alertasInsumos": [
    {
      "ingrediente": "Carne de Res Asada",
      "estado": "CRÍTICO",
      "mensaje": "Stock por debajo del 10%. Riesgo de desabastecimiento."
    }
  ]
}
```

11. Control de Errores Estandarizado (Global Exception Handler)

Para evitar que Spring Boot exponga trazas de error internas (StackTraces) que comprometan la seguridad, y para que Angular pueda pintar alertas amigables al usuario, todas las respuestas de error seguirán el estándar RFC 7807 (Problem Details):

- Ejemplo de Error (400 Bad Request – Producto Agotado):

```
{
  "type": "/errors/producto-agotado",
  "title": "Insumo No Disponible",
  "status": 400,
  "detail": "La 'Quesadilla de Birria' no puede ser procesada porque el ingrediente principal se agotó en cocina.",
  "instance": "/api/v1/pedidos"
```

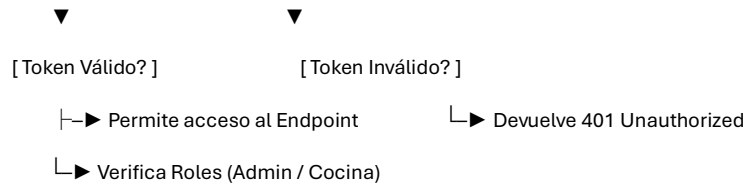
}

12. Diseño de Seguridad y Control de Acceso

12.1. Arquitectura de Seguridad

La API del backend operará bajo un modelo Stateless (sin estado). En lugar de usar sesiones de servidor que consumen memoria RAM innecesaria, cada petición a endpoints protegidos deberá incluir un token JWT firmado digitalmente en las cabeceras HTTP.

[Angular App] —(Envía JWT en cabecera Authorization)—► [Spring Security Filter Chain]



12.2. Modelo de Roles y Permisos (RBAC)

Definiremos tres roles claros dentro del sistema. La seguridad se aplicará directamente a nivel de los controladores de Spring Boot usando anotaciones @PreAuthorize.

ROL	ORIGEN DE ACCESO	PERMISOS CLAVES	ENDPOINT PERMITIDOS	
ANONYMOUS / CLIENTE	Escaneo de QR (Celular) Consultar menú	Crear pedidos.	GET /api/v1/productos POST /api/v1/pedidos	
COCINA	Tablet en cocina Cambiar estado de pedidos	Ver KDS	SUBSCRIBE /topic/cocina/** PATCH /api/v1/pedidos/{id}/estado	
ADMINISTRADOR	PC de Escritorio / Dashboard Control total: modificar precios, ver analíticas de IA, gestión de stock.	Gestión de stock	ALL /api/v1/admin/** ALL /api/v1/**	

12.3. Configuración de CORS (Cross-Origin Resource Sharing)

Como nuestra aplicación de Angular se ejecutará en un puerto o dominio distinto al del backend de Spring Boot (por ejemplo, Angular en el puerto 4200 y Spring Boot en el 8080), el backend debe autorizar explícitamente al frontend.

En la clase CorsConfig de Spring Boot, se restringirán los accesos de la siguiente manera:

Allowed Origins: Solo se permitirán los dominios oficiales de las interfaces (ej: http://localhost:4200 en desarrollo o el dominio de producción).

Allowed Methods: GET, POST, PATCH, OPTIONS (se bloquean DELETE y PUT no autorizados).

Allowed Headers: Authorization, Content-Type, X-Requested-With.

12.4. Flujo de Autenticación para Personal (JWT)

Para que el administrador o el cocinero entren a sus paneles en Angular, se ejecutará el siguiente ciclo técnico:

- Autenticación (Login): Angular envía las credenciales vía POST /api/v1/auth/login.
- Generación del Token: Spring Boot valida contra la base de datos (usando contraseñas encriptadas con BCryptPasswordEncoder) y, si todo es correcto, genera un JWT firmado con un algoritmo seguro (HS256).
- Estructura del Token: El backend responde con el token, el cual lleva codificado los roles del usuario dentro de sus reclamos (claims):

```
{  
  "tokenType": "Bearer",  
  "accessToken":  
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ0YXF1ZXJvXzAxliwicm9sZXMiOiUk9MRV9DT0NJTkEiXSwiaWF0IjoxNzE3MjU0NDAwLCJleHAiOiE3MTczNDA4MDB9...",  
  "expiracionSegundos": 86400  
}
```

Persistencia en Frontend: Angular intercepta esta respuesta a través de un AuthService, almacena el token en la memoria del navegador o localStorage de forma segura, y lo inyecta automáticamente en cada petición HTTP futura usando un HttpInterceptor.

12.5. Seguridad en WebSockets (STOMP Security)

Los WebSockets no usan las cabeceras HTTP estándar después del saludo inicial (handshake). Por lo tanto, configuraremos un ChannelInterceptor en Spring Boot.

Cuando la pantalla de cocina en Angular intente suscribirse a /topic/cocina/pedidos, el interceptor de Spring Boot extraerá el token JWT enviado en las cabeceras nativas de STOMP. Si el token no tiene el rol ROLE_COCINA o ROLE_ADMIN, el backend cerrará la conexión inmediatamente, evitando filtraciones de pedidos en tiempo real.

13. Diseño Base de Datos Relacional (PostgreSQL 16)

13.1. Modelo Entidad – Relación Físico

A continuación, se detalla la estructura DDL (Data Definition Language) optimizada para producción, incluyendo restricciones de integridad (NOT NULL, CHECK) e índices para acelerar las consultas en las horas pico.

- Habilitar la extensión para generación de UUID nativos si no está activa

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

- 1. TABLA: CATEGORIAS

```
CREATE TABLE categorias (
```

```
  Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
  Nombre VARCHAR(50) NOT NULL UNIQUE,
```

```
  Activo BOOLEAN DEFAULT TRUE NOT NULL,
```



```
Created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL  
);
```

■ 2. TABLA: PRODUCTOS (Menú)

```
CREATE TABLE productos (  
  Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  Categoria_id UUID NOT NULL,  
  Nombre VARCHAR(100) NOT NULL,  
  Descripcion TEXT,  
  Precio NUMERIC(10, 2) NOT NULL CHECK (precio >= 0),  
  Imagen_url VARCHAR(255),  
  Disponible BOOLEAN DEFAULT TRUE NOT NULL,  
  Created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,  
  CONSTRAINT fk_productos_categorias FOREIGN KEY (categoria_id) REFERENCES categorias(id) ON DELETE RESTRICT  
);
```

■ 3. TABLA: PEDIDOS (Cabecera de la Orden)

```
CREATE TABLE pedidos (  
  Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  Codigo_orden SERIAL NOT NULL, -- Número corto secuencial diario para el ticket del cliente y la cocina  
  Nombre_cliente VARCHAR(100) NOT NULL,  
  Telefono_cliente VARCHAR(20) NOT NULL,  
  Identificador_mesa VARCHAR(20) NOT NULL, -- Capturado automáticamente del QR (Ej: 'Mesa-3')  
  Nota_cocina TEXT,  
  Total_netto NUMERIC(10, 2) NOT NULL CHECK (total_netto >= 0),  
  Metodo_pago VARCHAR(30) NOT NULL CHECK (metodo_pago IN ('EFFECTIVO_CAJA', 'TARJETA_ONLINE',  
  'QR_TRANSFERENCIA')),  
  Estado_actual VARCHAR(30) DEFAULT 'PENDIENTE' NOT NULL CHECK (estado_actual IN ('PENDIENTE', 'PAGADO',  
  'PREPARANDO', 'LISTO', 'ENTREGADO')),  
  Created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL  
);
```

■ 4. TABLA: DETALLES_PEDIDO (Relación N:N Desglosada)

```
CREATE TABLE detalles_pedido (  

```

```

Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

Pedido_id UUID NOT NULL,

Producto_id UUID NOT NULL,

Cantidad INTEGER NOT NULL CHECK (cantidad > 0),

Precio_unitario NUMERIC(10, 2) NOT NULL CHECK (precio_unitario >= 0), -- Captura el precio histórico del momento de la compra

Subtotal NUMERIC(10, 2) GENERATED ALWAYS AS (cantidad * precio_unitario) STORED NOT NULL,

CONSTRAINT fk_detalle_pedido FOREIGN KEY (pedido_id) REFERENCES pedidos(id) ON DELETE CASCADE,

CONSTRAINT fk_detalle_producto FOREIGN KEY (producto_id) REFERENCES productos(id) ON DELETE RESTRICT

);

```

■ 5. TABLA: HISTORIAL_ESTADOS (Auditoría para la IA)

```

CREATE TABLE historial_estados_pedido (

Id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

Pedido_id UUID NOT NULL,

Estado VARCHAR(30) NOT NULL,

Fecha_cambio TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,

CONSTRAINT fk_historial_pedido FOREIGN KEY (pedido_id) REFERENCES pedidos(id) ON DELETE CASCADE

);

```

13.2. Estrategia de Indexación (Optimización de Rendimiento)

Para evitar que el sistema se vuelva lento cuando el histórico de pedidos crezca a miles de registros, crearemos Índices Secundarios (B-Tree) en las columnas estratégicas que usará la aplicación:

■ Acelera la carga de tarjetas en el KDS de cocina filtrando por estado_actual

```
CREATE INDEX idx_pedidos_estado ON pedidos(estado_actual);
```

■ Optimiza las consultas del Dashboard de Administración por rango de fechas (Ventas de hoy)

```
CREATE INDEX idx_pedidos_fecha ON pedidos(created_at);
```

■ Optimiza la búsqueda de detalles pertenecientes a una orden específica

```
CREATE INDEX idx_detalle_pedido_id ON detalles_pedido(pedido_id);
```

14. Mapeo en el Backend (Ejemplo de Entidad JPA en Java 17)

Para demostrar la "alta compatibilidad" configurada, así se estructuraría la clase Entidad en Spring Boot utilizando Lombok y tipos de datos modernos:

```
Package com.tacos.api.domain.model;
```

```
Import jakarta.persistence.*;

Import lombok.*;

Import java.math.BigDecimal;

Import java.time.OffsetDateTime;

Import java.util.UUID;
```

```
@Entity
```

```
@Table(name = "pedidos")
```

```
@Getter
```

```
@Setter
```

```
@NoArgsConstructor
```

```
@AllArgsConstructor
```

```
@Builder
```

```
Public class Pedido {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.UUID)
```

```
    Private UUID id;
```

```
    @Column(name = "codigo_orden", insertable = false, updatable = false)
```

```
    Private Integer codigoOrden;
```

```
    @Column(name = "nombre_cliente", nullable = false)
```

```
    Private String nombreCliente;
```

```
    @Column(name = "telefono_cliente", nullable = false)
```

```
    Private String telefonoCliente;
```

```
    @Column(name = "identificador_mesa", nullable = false)
```

```
    Private String identificadorMesa;
```

```
    @Column(name = "nota_cocina")
```

```
    Private String notaCocina;
```

```
@Column(name = "total_neto", nullable = false)
```

```
Private BigDecimal totalNeto;
```

```
@Column(name = "metodo_pago", nullable = false)
```

```
Private String metodoPago;
```

```
@Column(name = "estado_actual", nullable = false)
```

```
Private String estadoActual;
```

```
@Column(name = "created_at", nullable = false, updatable = false)
```

```
Private OffsetDateTime createdAt;
```

```
}
```